

Advanced C# & .NET Cheatsheet

Modern Language Features, Runtime Architecture, Web APIs & Coding Snippets

ADVANCED EDITION

1. AGENDA & MODULE ROADMAP

Page	Module	Topics Covered
Page 1	Modern Language Features	Records, Immutability, Advanced Pattern Matching, Nullability, Extension Methods
Page 2	Runtime & Functional C#	Generational GC, IDisposable, Delegates, Func/Action, Events, String Interning
Page 3	EF Core & ASP.NET Core	LINQ Syntax Comparison, EF Core Execution, DI Lifetimes, RESTful Controllers
Page 4	Patterns & Coding Round	Singleton (`Lazy`), Factory Pattern, Top 5 Coding Snippets (Reverse, Palindrome, etc.)

2. RECORDS & IMMUTABILITY (C# 9+)

Records are reference types that use **value-based equality**. Ideal for Data Transfer Objects (DTOs).

Positional Record Definition

```
// Immutable by default with init-only properties
public record Person(string Name, int Age);

var p1 = new Person("Alice", 25);
var p2 = new Person("Alice", 25);

Console.WriteLine(p1 == p2); // True!
(Value-based equality)
```

Nondestructive Mutation (`with`)

```
var original = new Person("Bob", 30);

// Creates a new copy with modified Age
var updated = original with { Age = 31 };

Console.WriteLine(updated.Name); // "Bob"
Console.WriteLine(updated.Age); // 31
```

3. PATTERN MATCHING & NULL HANDLING

Property & Relational Patterns

```
string GetDiscount(object obj) => obj
switch
{
    Person { Age: >= 65 } => "Senior
Discount",
    Person { Age: < 18 } => "Youth
Discount",
    null => "No Customer",
    _ => "Standard
Rate"
};
```

Modern Null Operators

```
string? text = null;

// Null-coalescing (??) & Assignment (??=)
string display = text ?? "Default";
text ??= "Initialized";

// Null-conditional (?)
int? length = text?.Length;
```

4. EXTENSION METHODS

Add new methods to existing types without modifying or inheriting them. Must be in a static class.

```
public static class StringExtensions {
    public static bool IsCapitalized(this string str) {
        if (string.IsNullOrEmpty(str)) return false;
    }
}
```

5. GARBAGE COLLECTION & RESOURCE MANAGEMENT

The .NET Garbage Collector (GC) manages heap memory using three generations:

Generation	Lifecycle & Scope	GC Frequency & Cost
Generation 0	Short-lived objects (e.g., local variables, temp buffers).	Cleaned very frequently; extremely fast.
Generation 1	Buffer zone for objects that survived Gen 0 collection.	Medium frequency cleanup.
Generation 2	Long-lived objects (e.g., Singletons, static data, LOH).	Collected during Full GC; can cause performance pauses.

Unmanaged Resources & IDisposable Pattern

```
// Deterministic disposal of unmanaged resources (e.g., db connections, files)
public class FileManager : IDisposable {
    public void Dispose() {
        // Clean up unmanaged resources here
    }
}

// 'using' statement guarantees Dispose() call even if exceptions occur
using var manager = new FileManager();
```

6. DELEGATES, FUNC, ACTION & EVENTS

Func<T> vs Action<T>

```
// Action: No return value (void)
Action<string> log = msg =>
Console.WriteLine(msg);
log("Processing...");

// Func: Last type argument is return type
Func<int, int, int> multiply = (x, y) => x
* y;
int product = multiply(4, 5); // 20
```

Events & Publisher Pattern

```
public class Button {
    // Event encapsulates delegate access
    public event Action? OnClick;

    public void Click() =>
OnClick?.Invoke();
}

// Subscribing to event: btn.OnClick += ()
=> ...;
```

7. STRING INTERNING & STRINGBUILDER

String Intern Pool

```
// Strings are immutable. Equal literals
share memory
string s1 = "Hello";
string s2 = "Hello";

bool sameRef = object.ReferenceEquals(s1,
s2);
// True! Stored in runtime String Intern
Pool
```

StringBuilder (Mutable String)

```
// Avoids creating intermediate objects in
loops
var sb = new System.Text.StringBuilder();
for (int i = 0; i < 1000; i++) {
    sb.Append(i);
}
string result = sb.ToString();
```

8. LINQ SYNTAX & EF CORE EXECUTION

Method Syntax vs. Query Syntax

```
// Method Syntax (Preferred in practice)
var res1 = users.Where(u => u.IsActive)
                .OrderBy(u => u.Name);

// Query Syntax (SQL-like)
var res2 = from u in users
           where u.IsActive
           orderby u.Name
           select u;
```

Deferred vs. Immediate Execution

```
// Deferred: Query NOT executed yet
// (IQueryable)
var query = dbContext.Users.Where(u =>
    u.Age > 18);

// Immediate: Executes SQL on database
// RIGHT NOW
List<User> list = query.ToList();
User? user = query.FirstOrDefault();
```

9. ASP.NET CORE DEPENDENCY INJECTION (DI) LIFETIMES

Service Lifetime	Instance Behavior	Best Use Case
Transient AddTransient	A new instance is created every time it is requested.	Lightweight, stateless services.
Scoped AddScoped	Created once per HTTP Request cycle. Shared within same request.	Database contexts (`DbContext`), Unit of Work.
Singleton AddSingleton	Created once at application startup and shared globally.	Application state, caching, single configuration clients.

10. RESTFUL API CONTROLLER ARCHITECTURE

```
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase {
    private readonly IProductService _service;

    // Dependency Injection via Constructor
    public ProductsController(IProductService service) => _service = service;

    [HttpGet("{id:int}")]
    public async Task<ActionResult> GetById(int id) {
        var product = await _service.GetProductByIdAsync(id);
        if (product == null) return NotFound();
        return Ok(product);
    }
}
```

11. ESSENTIAL DESIGN PATTERNS

Thread-Safe Singleton (`Lazy`)

```
public class LoggerService {
    private static readonly
    Lazy<LoggerService> _instance =
        new(() => new LoggerService());

    private LoggerService() { }
    public static LoggerService Instance =>
        _instance.Value;
}
```

Factory Method Pattern

```
public static class NotificationFactory {
    public static INotification
    Create(string type) => type switch {
        "Email" => new EmailNotification(),
        "SMS"   => new SmsNotification(),
        _       => throw new
        ArgumentException("Invalid type")
    };
}
```

12. TOP 5 CODING ROUND SNIPPETS FOR FRESHERS

1. Reverse a String Without Built-ins

```
public string Reverse(string input) {
    char[] chars = input.ToCharArray();
    int left = 0, right = chars.Length - 1;
    while (left < right) {
        (chars[left], chars[right]) =
        (chars[right], chars[left]);
        left++; right--;
    }
    return new string(chars);
}
```

2. Check for Palindrome

```
public bool IsPalindrome(string s) {
    int i = 0, j = s.Length - 1;
    while (i < j) {
        if (char.ToLower(s[i]) !=
        char.ToLower(s[j]))
            return false;
        i++; j--;
    }
    return true;
}
```

3. Character Frequency Count

```
public Dictionary<char, int>
GetFrequency(string input) {
    var map = new Dictionary<char, int>();
    foreach (char c in input) {
        map[c] = map.GetValueOrDefault(c,
        0) + 1;
    }
    return map;
}
```

4. Find Duplicate with HashSet

```
public bool HasDuplicates(int[] nums) {
    var set = new HashSet<int>();
    foreach (int num in nums) {
        if (!set.Add(num)) return true; //
        Add fails if duplicate
    }
    return false;
}
```

5. Find Missing Number in Array [1..N]

```
public int FindMissingNumber(int[] nums, int n) {
    int expectedSum = n * (n + 1) / 2;
    int actualSum = nums.Sum();
    return expectedSum - actualSum;
}
```